

КАК УСТАНОВИТЬ РОБОТА НА СВОЕМ КОМПЬЮТЕРЕ И НАЧАТЬ РАБОТУ

Установка Julia и VS Code

1. Установите **Julia**, запустив установщик с [официального сайта](#)
2. Установите интегрированную среду разработки **VS Code** с [официального сайта](#)
3. Откройте **VS Code** и установите расширение для поддержки языка **Julia**
4. Запустите **Julia** из командной строки **VS Code** - в результате появится командная строка интерактивной среды (**REPL Julia**)

Установка пакетов GLMakie и HorizonSideRobots

1. Переведите командную строку **Julia** в режим пакетного менеджера нажатием клавиши **]**
2. Из командной строки пакетного менеджера запустите установку пакета **GLMakie**:

```
pkg> add GLMakie
```

3. После завершения установки **GLMakie** из той же командной строки пакетного менеджера установите пакет **HorizonSideRobots**:

```
pkg> add https://github.com/Vibof/HorizonSideRobots.jl
```

Здесь понадобилось указать полный URL, потому что пакет **HorizonSideRobots** в отличие от пакета **GLMakie** не является зарегистрированным.

4. После завершения установки из режима пакетного менеджера вернитесь в обычный режим командной строки **REPL Julia** нажатием клавиши **Backspace**

Импортирование установленных пакетов и первый запуск Робота

Если все прошло нормально, то установленные пакеты могут быть импортированы, например, непосредственно в интерактивную среду **REPL Julia**:

```
julia> using GLMakie  
julia> using HorizonSideRobots
```

(последовательность импортов значения не имеет).

Важно, чтобы среда **VS Code** была настроена таким образом, чтобы графика выводилась в отдельном окне, а не в специальной встроенной панели для графиков.

После этого Робота можно будет запустить непосредственно из командной строки интерактивной среды:

```
julia> r = Robot(animate = true)
```

В результате выполнения этой команды откроется окно с Роботом на клетчатом поле, ограниченном прямоугольной рамкой. Переменной **r** при этом будет присвоено значение объекта типа **Robot** (это значение представляет собой ссылку).

Важно отметить, что в режиме визуализации пакет рассчитан на работу только с одним отображаемым Роботом одновременно.

Значение именованного аргумента конструктора **animate = true** означает, что Робот создается в режиме анимации, т.е. в режиме позволяющем визуально наблюдать его перемещения по полю. Соответственно, в случае **animate = false** наблюдать Робота и его перемещения будет невозможно.

Запускать Робота без визуализации следует, например, когда требуется исключить временные задержки, связанные с визуализацией, т.к. при достаточно больших размерах поля эти задержки могут оказаться весьма значительными.

С другой стороны, режим с визуализацией удобен при отладке, т.к. в этом случае имеется возможность визуально наблюдать пошаговое выполнение программы.

Создание поля произвольных размеров

Размеры поля можно задать произвольными, например, поле 20 на 25 клеток можно создать командой:

```
julia> r = Robot(20, 25, animate = true)
```

Редактирование обстановки на поле с помощью мыши и сохранение ее в файле

С помощью мыши на поле можно создать произвольную начальную обстановку.

Так, Робота можно переместить в любую видимую клетку поля. Для этого надо сначала кликнуть по телу робота - его цвет изменится на более светлый, затем кликнуть по целевой клетке - робот автоматически переместится в требуемую клетку.

Кликавая мышью по центру любой клетки, можно установить/убрать маркер, кликая вблизи границы между соседними клетками, можно установить/убрать перегородку между этими клетками.

Если кликнуть в любом месте вблизи внешней рамки поля, то можно убрать/восстановить внешнюю границу поля. При снятии внешней ограничивающей рамки поле становится бесконечным во все стороны. Это означает, что Робот сможет перемещаться за пределы видимой части поля, а потом возвращаться назад. При этом, если какая-либо внутренняя перегородка будет доходить вплотную до границы видимой части поля, то считается, что эта граница прямолинейно продолжается до бесконечности в соответствующую сторону.

Сразу после первого клика мышью для изменения обстановки на поле автоматически создается файл `untitled.sit`, в котором сохраняется создаваемая обстановка. В последствии этот файл можно переименовать и сохранить, например, для тестирования программы.

Справочная система Julia

Для получения справочной информации о командном интерфейсе Робота можно непосредственно в интерактивной среде `Julia`, воспользовавшись встроенной справочной системой. Для этого сначала надо перейти в соответствующий режим командной строки, выполнив команду:

```
julia> ?
```

в результате чего приглашение командной строки вид:

```
help?>
```

и далее:

```
help?> Robot
search: Robot

r = Robot(;animate=false)

r = Robot(num_rows::Integer, num_colons::Integer; animate=false)

r = Robot(file_name::AbstractString; animate=false)

- Конструктор объектов типа Robot (исполнитель "Робот на клетчатом поле со
сторонами горизонта типа HorizonSide"):

. - в первом случае - пустое ограниченное поле будет иметь размер 11x12 и
Робот будет - в левом нижнем углу

. - во втором случае - поле будет иметь указанные размеры

. - в третьем случае - обстановка загружается из указанного файла (обычно - с
расширением sit)

- Если animate=true, то при этом открывается окно с соответствующей начальной
```

обстановкой (которую можно будет редактировать, и результат редактирования будет сохраняться в файле `"untitled.sit"`). Иначе просто создается объект, содержащий данные, определяющие эту обстановку (визуализации обстановки в этом случае нет).

Командный интерфейс исполнителя (объекта) типа `Robot`: `move!`, `isborder`, `putmarker`, `ismarker`, `temperature`, `show_sit`, `show_sit!` (см. `help`)

Для подготовительной работы с `sit`-файлами (в них сохраняется данные с информацией о некоторой обстановке на поле с Роботом) имеются специальные функции: `show_sit!`, `edit_sit`, `create_sit` (см. `help`)

Для возможности получения этой информации предварительно пакет `HorizonSideRobots` должен быть импортирован.

Для получения информации о назначении и способах использования всех импортированных из пакета функций можно также воспользоваться встроенной системой помощи, например:

```
help?> move!  
search: move! merge!
```

```
move!(r::Robot, side::HorizonSide)::Nothing
```

– Перемещает Робота в соседнюю клетку в заданном направлении (если только на пути нет перегородки, в противном случае - прерывание)

(важно отметить, что `!` в конце имени некоторых функций - это часть имени каждой из таких функций).

Запуск программы и работа со скриптами

Хотя любое действие можно выполнять непосредственно из командной строки интерактивной среды `Julia`, программный код, который предназначен для многократного исполнения или содержит значительное число строк, следует размещать в `j1`-файлах. Эти файлы следует помещать в отдельную рабочую папку, которая должна быть открыта с помощью встроенного файлового менеджера `VS Code` (в этом случае рабочая папка становится текущей для интерактивной командной строки).

Для удобства рекомендуется создать файл `start.j1` со скриптом:

```
# start.j1  
  
using GLMakie  
using HorizonSideRobots
```

```
robot = Robot(animate = true)
```

Тогда, если этот файл находится в открытой рабочей папке, то из командной строки данный скрипт может быть выполнен с помощью стандартной библиотечной функции `include`:

```
julia> include("start.jl")
```

Это равнозначно тому, как если бы команды содержащиеся в скрипте вводились бы непосредственно в командной строке.

Однако, если имеется заранее созданный файл `test-1.sit` с некоторой начальной обстановкой на поле, то скрипт `start.jl` можно было бы написать так:

```
# start.jl

using GLMakie
using HorizonSideRobots

const START_SIT = "test-1.sit"
const ANIMATE = true

robot = Robot(START_SIT, animate = ANIMATE)
```

В каждом новом случае значения определяемых здесь глобальных констант `START_SIT` и `ANIMATE` могут быть отредактированы требуемым образом. Кроме того, сохранение начальной обстановки в файле позволяет автоматизировать тестирование и обеспечивает воспроизводимость работы тестируемой программы.

В случае, когда начальная обстановка создана заранее и сохранена в файле, функция `include("start.jl")` может помещаться уже сразу в основной файл вместе с некоторым программным кодом. Например, содержимое некоторого файла `test-1.jl` может выглядеть так:

```
# test_1.jl

include("start.jl") # -> robot - объект типа Rrobot с заданной начальной
обстановкой
include("lib_rob.jl") # -> prog - некоторый пользовательский код
prog(robot)
```

где `lib_rob.jl` - это ещё один файл, содержащий определение некоторой пользовательской функции `prog`, в которую ссылка на созданный в `start.jl` объект типа `Robot`, содержащаяся в переменной `robot`, передается в функцию `prog` как аргумент.

Здесь предполагается, что файл `lib_rob.jl` находится в рабочей папке рядом с остальными файлами (либо должен быть указан полный путь к этому файлу).

В результате, запуск скрипта `test-1.jl` может быть осуществлен, например, так:

```
julia> include("test-1.jl")
```